



BLACK DUCK[®]

TRUE SCALE APPLICATION SECURITY

From Prompt to Governance

Engineering Trust in AI-Generated Code

HUGO YANG

JULY 2026



CONFIDENTIAL INFORMATION

The information contained in this presentation is the confidential and proprietary information of Black Duck Software. You are not permitted to disseminate or use any of the information provided to you in this presentation outside of Black Duck Software without prior written authorization.

IMPORTANT NOTICE

In the event information in this presentation reflects Black Duck Software's future plans, such plans are as of the date of this presentation and are subject to change. Black Duck Software is not obligated to update this presentation or develop the products with the features and functionality discussed in this presentation. Additionally, Black Duck Software's services and products may only be offered and purchased pursuant to an authorized quote and purchase order or a mutually agreed upon written contract with Black Duck Software.



Presented to you by

- Hugo YANG
- Black Duck Asia-Pacific (Singapore)
 - Engineering Principal
 - APAC Ecosystem Enabler
- The content in this presentation is primarily authored by me, with less than 20% AI assistance





On 21 May, I prompted GPT 5.2 to create an agenda flow for today's sharing:

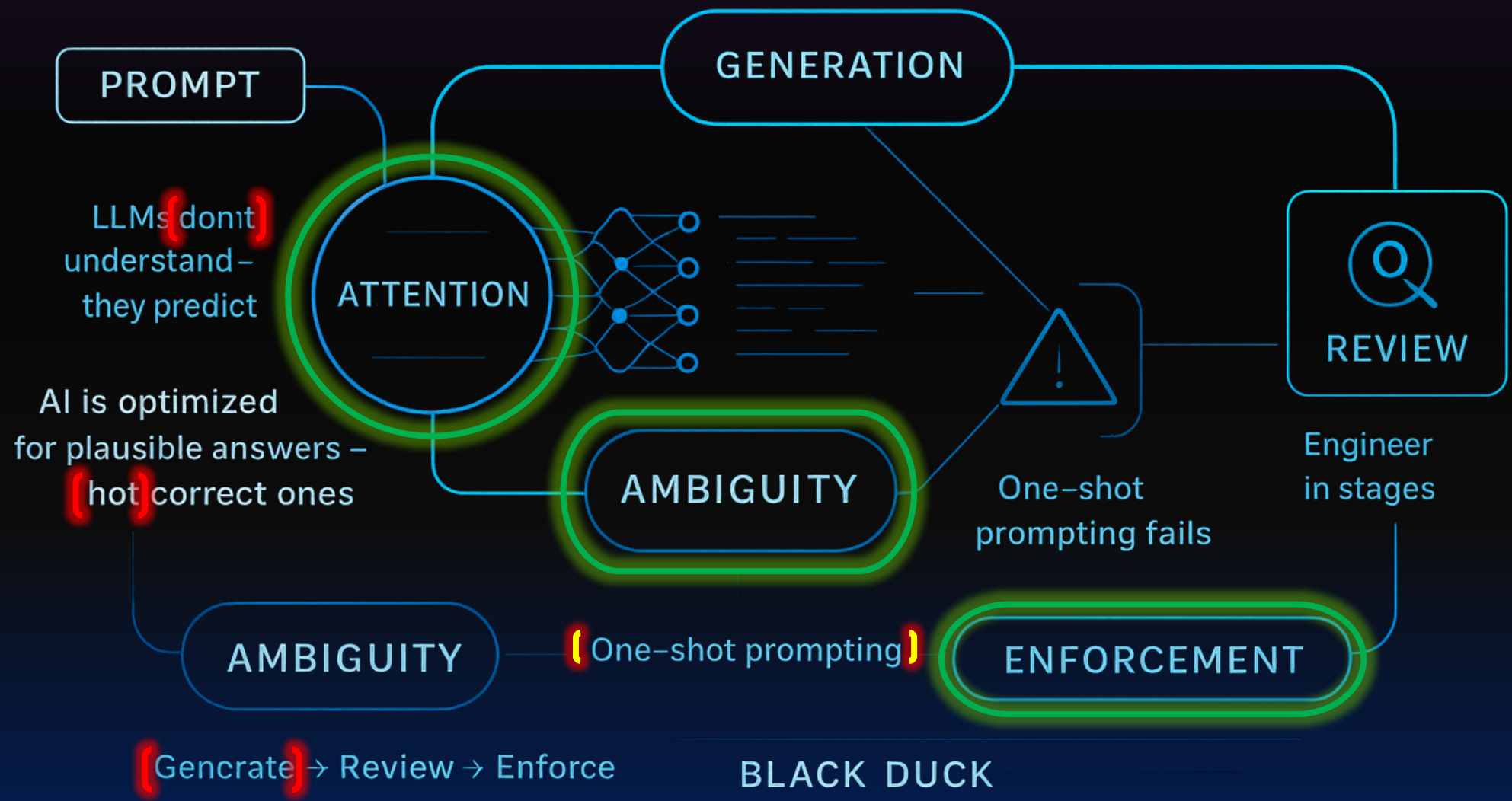
agenda/tight summary (/my prompt)

` I'll start by demystifying how LLMs process prompts using the transformer attention mechanism — not as 'understanding,' but as context-weighted next-token prediction. From there, I'll explain why outputs are inherently probabilistic and why prompt complexity increases ambiguity and failure modes. That sets up a practical engineering pattern: use AI for generation in focused steps, then verify with AI-assisted review and deterministic scanning — rather than relying on one-shot prompts. Finally, I'll show how Black Duck provides guardrails across the workflow: rule-based scanning in the IDE and repository, plus LLM-powered agentic review for broader language coverage and deeper semantic checks — enabling secure, compliant AI-assisted software delivery. `



And this is what I got, 80% good, but there are errors:

From Prompt to Trust: The Journey of AI Code





A realistic AI-assisted dev workflow at high level

1. The developer provides context

2. Input is tokenized and embedded into vector representations

3. Transformer layers apply attention to compute contextual relationships

4. Model generates code output via probabilistic next-token prediction

5. Developer reviews and iterates

6. Governance & Security Harness Readiness

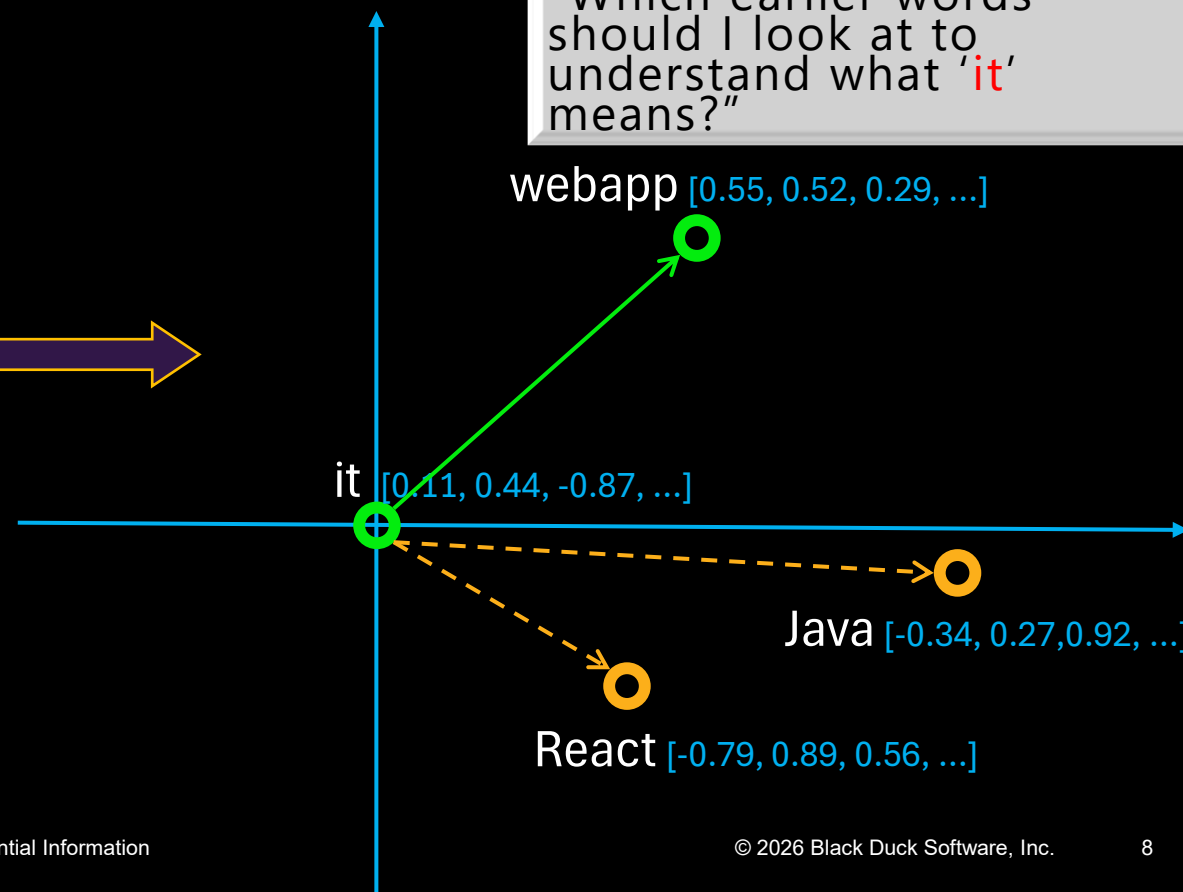
Input -> Understanding *Attention Is All You Need*

When we prompt to code

help me with a webapp using Java and React, it must be...

#1 problem for models:
"Which earlier words
should I look at to
understand what '*it*'
means?"

Client Input		LLM Models
Tokens		Embedding Vector
• help	◆-----	[0.21, -0.45, 0.92, ...]
• ...	◆-----	[...]
• webapp	◆-----	[0.55, 0.52, 0.29, ...]
• Java	◆-----	[-0.34, 0.27, 0.92, ...]
• React	◆-----	[-0.79, 0.89, 0.56, ...]
• It	◆-----	[0.11, 0.44, -0.87, ...]
• must	◆-----	[-0.99, 0.77, -0.75, ...]
• ...	◆-----	[...]



Input -> Probability Computation

Models don't rely on common sense; they rely on **mathematics**.

Models generate query vector

Similarity Scoring	Token	Attention weight
--------------------	-------	------------------

Query(it) × Key(webapp)	webapp	0.7
-------------------------	--------	-----

Query(it) × Key(Java)	Java	0.2
-----------------------	------	-----

Query(it) × Key(React)	React	0.1
------------------------	-------	-----

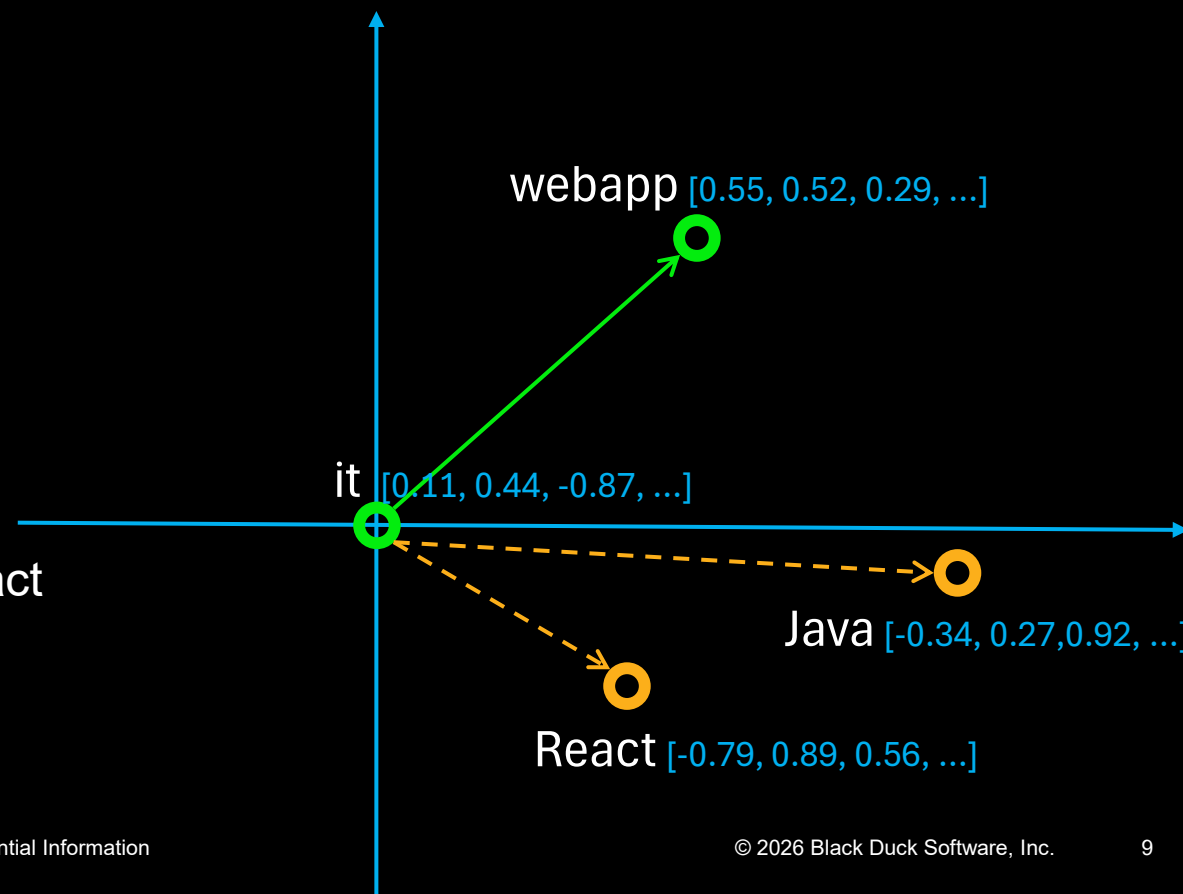


$it_vector = \sum (attention_weight \times value_vector)$
or simplified as

$It_embedding \approx 0.7 \times webapp + 0.2 \times Java + 0.1 \times React$



“it” = the web application, not Java or React





Attention mechanism functions as a dynamic highlighting process. When the model processes “it,” it retrospectively scans approximately 110 words (~180 tokens) in the prompt, computing token relevance using Query–Key similarity and applying softmax normalization to derive context-dependent attention weights.

Complex prompts increase risk because they create:

- Competing constraints (conflicts inside the prompt)
- Long-range dependencies (early requirements get “forgotten”)
- Ambiguity (multiple valid interpretations)
- Hidden assumptions (unstated environment, libraries, policies)

Help me design and implement a production-grade web application using Java (Spring Boot) for the backend and React (TypeScript) for the frontend. The application must follow clean architecture principles with clear separation of concerns, and be built with security by design. It should include secure authentication (OAuth2), role-based access control, proper input validation, and protections against OWASP Top 10 risks such as XSS, SQL injection, and CSRF. The backend should expose well-structured REST APIs, use secure coding practices (e.g., parameterized queries, input sanitization), and integrate with a relational database like PostgreSQL via JPA/Hibernate. The frontend should implement secure token handling, avoid storing sensitive data in the browser, and it shall follow modern React patterns (hooks, functional components)

Drifts in probability estimation propagate into the final output, resulting in incorrect predictions. Thus, **Security Vulnerabilities.**

To be fair to models, “the model made the best guess—but it was wrong”



Black Duck delivers a unified solution for human-driven, AI-enabled software development

Components of my solution

- On-prem and/or SaaS
- Agentic AI workflow and/or deterministic best-rule-based engine
- Secret detection
- Static Source Code Analysis
- Dependency Analysis
- 3rd Party License Obligation Detection
- Software Supply Chain Risk Mitigation
- Dynamic Testing

blackduck.com/

Select, integrate, and scale AppSec capabilities per your AI journey



Prompt

- IP & Secrete Leakage Defense (R&D projects)

Governance

- Compliance Reporting: CERT-Java/Autosar/OWASP...
- SBOM, VEX, and SARIF
- Bug tracking integration for future dataset collection
- PR comment/ Break-Build

Generate

- Stop insecure, or license violating, code reaching IDE (R&D projects)

PR/Repo/Staging Analysis

- Checker-driven, data flow-aware static analysis
- Software Composition Analysis
- Dynamic Testing on webapp and API endpoints
- Agentic AI Code Review

Validation @ IDE

- Rule-based rapid static analysis
- AI code license analysis
- Agentic AI powered Diff Scan



Rapid Security Validation @ IDE

➤ Deterministic App Security

- Security Static Scanning + Licenses compliance analysis
- Single file analysis completed within 2 mins (in most real-world practices)
- Solving the most immediate problems when the model is generating the output
- Low infrastructure, early-stage security feedback loop

➤ Agentic App Security

- Programming Language agnostic
- Analyzes source code directly — no compilation or build environment needed



PR/Repo/Staging Analysis

➤ Deterministic App Security

- Static and Runtime testing
- Compiles the code, traces every dataflow path — precision defect detection, deterministically.
- Secures the software supply chain — regardless of whether humans or AI chose the dependencies
- Fast feedback loop for enterprise-scale software development in the software dev pipeline

➤ Agentic App Security

- Semantic Security Review, reasons about code intent, not just structure,
able to detect business logic flaws



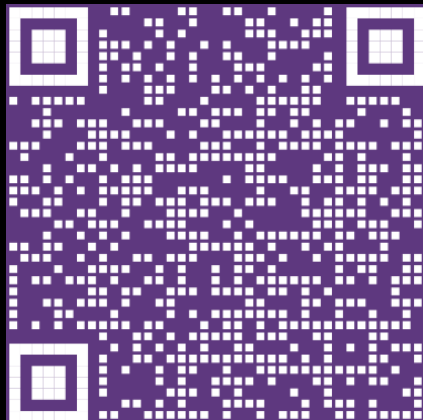
Governance: Harnessing Trust in AI-Assisted Development

- Enforce compliance against industry standards — CERT, AUTOSAR, OWASP, PCIDSS and more — with audit-ready reporting.
- AI-driven threat modeling ready
 - Generate machine-readable artifacts — SBOM for supply chain transparency, VEX for vulnerability communication, SARIF for tool-agnostic findings integration.
 - Integrate findings directly into bug tracking systems — building a living dataset for continuous security posture improvement.
- Shift security left — surface findings as PR comments for developer action, enforce policy gates to break the build before insecure code ships.

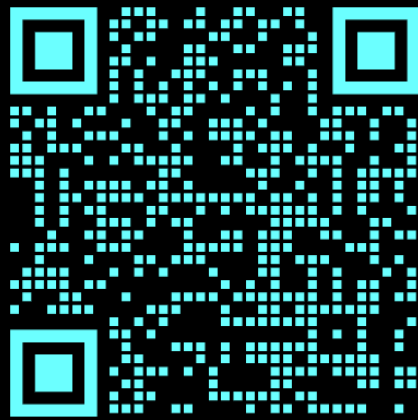


Get my datasheet

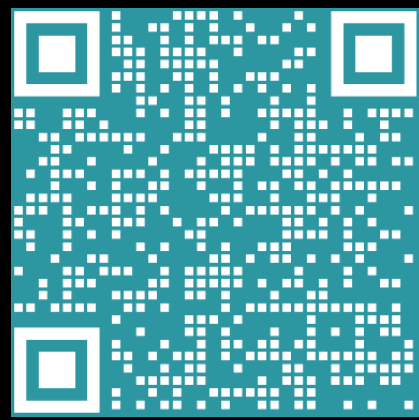
Signal Agentic AI
Application
Security



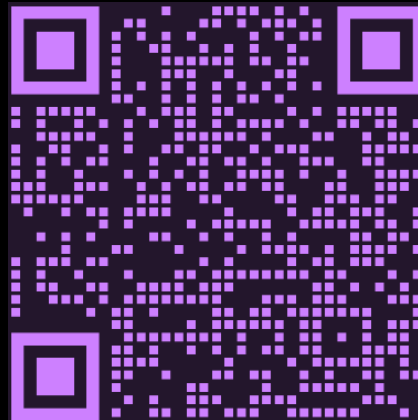
Coverity
Static Analysis



Black Duck
Software
Composition
Analysis



Polaris
Application
Security
Platform





Thank You